

UNIVERSIDADE PAULISTA - UNIP

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

GUILHERME AUGUSTO GATTO - R.A: R2798I3

ITÁLO RENAN SANTOS LIMA - R.A: R2792H1

JOÃO PEDRO FREITAS OLIVEIRA - R.A: H838339

LUIS HENRIQUE DE AQUINO MELLO - R.A: R2882I9

IGOR RIBEIRO MORAIS DA SILVA - R.A: R335011

PROJETO INTEGRADO MULTIDISCIPLINAR SÃO JOSÉ DOS CAMPOS 2026

SÃO JOSÉ DOS CAMPOS

2026

RESUMO

Este Projeto Integrado Multidisciplinar (PIM II) foi desenvolvido com base no planejamento de um sistema informatizado para a **Barbearia Santos**, um estabelecimento fictício criado para dar contexto prático ao nosso aprendizado acadêmico. A ideia principal do projeto é resolver problemas comuns do dia a dia de um pequeno negócio, substituindo o atendimento manual e a troca de mensagens por WhatsApp por uma ferramenta digital organizada e fácil de usar. O objetivo principal é aplicar de verdade os conhecimentos que estamos estudando nas matérias do segundo semestre do curso Superior de Análise e Desenvolvimento de Sistemas. Para tirar a ideia do papel, o projeto foi estruturado passo a passo, definindo quem pode acessar o quê no sistema — separando as funções do dono e do cliente. Com isso, o software foi planejado para permitir que os clientes façam seus próprios agendamentos e cadastros com autonomia, enquanto o proprietário consegue gerenciar os horários da barbearia, cadastrar novos serviços, acompanhar o estoque de produtos de forma simples e consultar o histórico de tudo o que já foi atendido. Dessa forma, o trabalho mostra como a tecnologia pode ser útil na rotina de uma empresa, trazendo mais organização para o fluxo de informações, segurança para os dados salvos e facilitando tanto a vida de quem administra quanto de quem consome o serviço.

Palavras-chave: Barbearia Santos, Análise e Desenvolvimento de Sistemas, Gestão de Agenda, Automação, Controle de Estoque.

RESUMO DE LINGUA ESTRANGEIRA (INGLÊS)

This Multidisciplinary Integrated Project (PIM II) was developed based on the planning of a computerized system for Barbearia Santos, a fictional business created to give practical context to our academic learning. The core idea of the project is to solve everyday challenges faced by small businesses by replacing manual service and WhatsApp text messaging with an organized, user-friendly digital tool.

The primary objective is to genuinely apply the knowledge we are studying across the courses of the second semester of the Higher Education degree in Systems Analysis and Development.

To bring the concept to life, the project was structured step-by-step, establishing clear access controls within the system—separating the functionalities for the owner and the customers. Consequently, the software was designed to allow clients to handle their own bookings and registrations autonomously, while enabling the owner to manage shop hours, register new services, easily monitor product inventory, and review the history of all completed appointments.

Ultimately, this work demonstrates how technology can benefit daily business operations, bringing greater organization to information flows, safeguarding stored data, and making life easier for both administration and customers.

Keywords: Barbearia Santos, Systems Analysis and Development, Appointment Management, Automation, Inventory Control.

SUMÁRIO

UNIVERSIDADE PAULISTA - UNIP	1
RESUMO	1
RESUMO DE LINGUA ESTRANGEIRA (INGLÊS)	3
INTRODUÇÃO	4
Objetivos	5
2.1 Objetivo geral	5
2.2 Objetivo específico.....	5
DESENVOLVIMENTO	6
3.1 Levantamento de Dados e Contextualização do Cliente.....	6
3.2 Engenharia de Requisitos	7
3.3 Modelagem de Processos com UML	8
3.4 Modelagem de Banco de Dados (brModelo)	9
3.5 Implementação do Algoritmo em C (Code::Blocks).....	10
4 CONSIDERAÇÕES FINAIS.....	11
5 REFERÊNCIAS	12
6 APÊNDICE – Código-Fonte Completo da Aplicação em C.....	12

INTRODUÇÃO

Com o avanço constante da tecnologia da informação, a automatização de processos tornou-se um fator crucial para a eficiência operacional e para a competitividade de empresas de diversos setores. No segmento de prestação de serviços, como o de barbearias e estéticas masculinas, a transição de métodos tradicionais e manuais para sistemas de gestão informatizados possibilita um controle mais preciso de operações diárias, agendamentos e cadastros de clientes.

Atualmente, a Barbearia Santos enfrenta desafios operacionais decorrentes do gerenciamento manual de suas rotinas. A ausência de um sistema centralizado para o registro de clientes, controle de serviços e organização de horários pode ocasionar retrabalho, inconsistência em dados cadastrais e perda de eficiência no atendimento ao público.

Diante desse cenário, este projeto tem como objetivo propor e desenvolver um sistema de gestão sob medida para a Barbearia Santos. A implementação do software visa otimizar os processos administrativos e operacionais, garantindo maior agilidade no cadastro de usuários, confiabilidade no armazenamento das informações e facilidade no acompanhamento dos serviços prestados.

Para contemplar os aspectos técnicos e práticos do desenvolvimento, este trabalho está estruturado em seções que abordam o levantamento de requisitos funcionais e não funcionais, a modelagem de processos através de diagramas UML, a estruturação do banco de dados relacional, a definição da arquitetura de redes e a implementação da lógica do sistema na linguagem de programação C.

Objetivos

2.1 Objetivo geral Desenvolver e documentar um sistema de gestão informatizado para a Barbearia Santos, automatizando os processos operacionais de cadastro de usuários e controle de agendamentos por meio de uma aplicação estruturada na linguagem C, integrada a uma arquitetura de banco de dados e modelo de redes adequado.

2.2 Objetivo específico

- Mapear os processos operacionais e realizar o levantamento de requisitos funcionais e não funcionais do estabelecimento.
- Modelar o fluxo do sistema e a estrutura de dados utilizando diagramas UML e o Modelo Entidade-Relacionamento (MER).
- Projetar a infraestrutura de redes necessária para garantir a segurança, conectividade e disponibilidade do sistema.
- Implementar a lógica de cadastro, validação e controle de acesso na linguagem de programação C.
- Aplicar boas práticas de Engenharia de Software Ágil para o gerenciamento do projeto e versionamento do código.

DESENVOLVIMENTO

3.1 Levantamento de Dados e Contextualização do Cliente

A Barbearia Santos é um estabelecimento comercial do ramo de estética e cuidados masculinos que busca modernizar sua gestão operacional. Atualmente, o controle dos atendimentos, o cadastro dos clientes e a agenda diária de serviços são realizados de forma manual ou com auxílio de ferramentas descentralizadas, o que gera inconsistências no fluxo de informações e limita o acompanhamento do crescimento do negócio.

Durante a etapa de levantamento de dados e análise do cenário atual, identificou-se a necessidade prioritária de implementar uma solução software centralizada. As principais dores observadas no ambiente operacional incluem:

- **Falta de padronização no cadastro:** Ausência de um histórico confiável de clientes e administradores do sistema.
- **Risco de perda de dados:** Registros em meio físico (cadernos/fichas) vulneráveis a extravios e erros de digitação.
- **Inexistência de controle de acesso:** Falta de níveis de permissão (autenticação) para separar o acesso administrativo das funções básicas do sistema.

A partir do mapeamento dessas necessidades, definiu-se o escopo inicial do software, priorizando na primeira etapa (Sprint 1) a implementação dos módulos de autenticação, cadastro e controle de acesso de usuários.

3.2 Engenharia de Requisitos

Com base na análise das necessidades operacionais da Barbearia Santos, foram mapeados os Requisitos Funcionais (RF), que especificam o comportamento e as funcionalidades oferecidas pelo software, e os Requisitos Não Funcionais (RNF), que definem os critérios de desempenho, segurança e usabilidade do sistema.

Para a **Sprint 1**, o escopo prioriza o controle de acesso, o cadastro de usuários e a verificação de credenciais, conforme detalhado no Quadro 1.

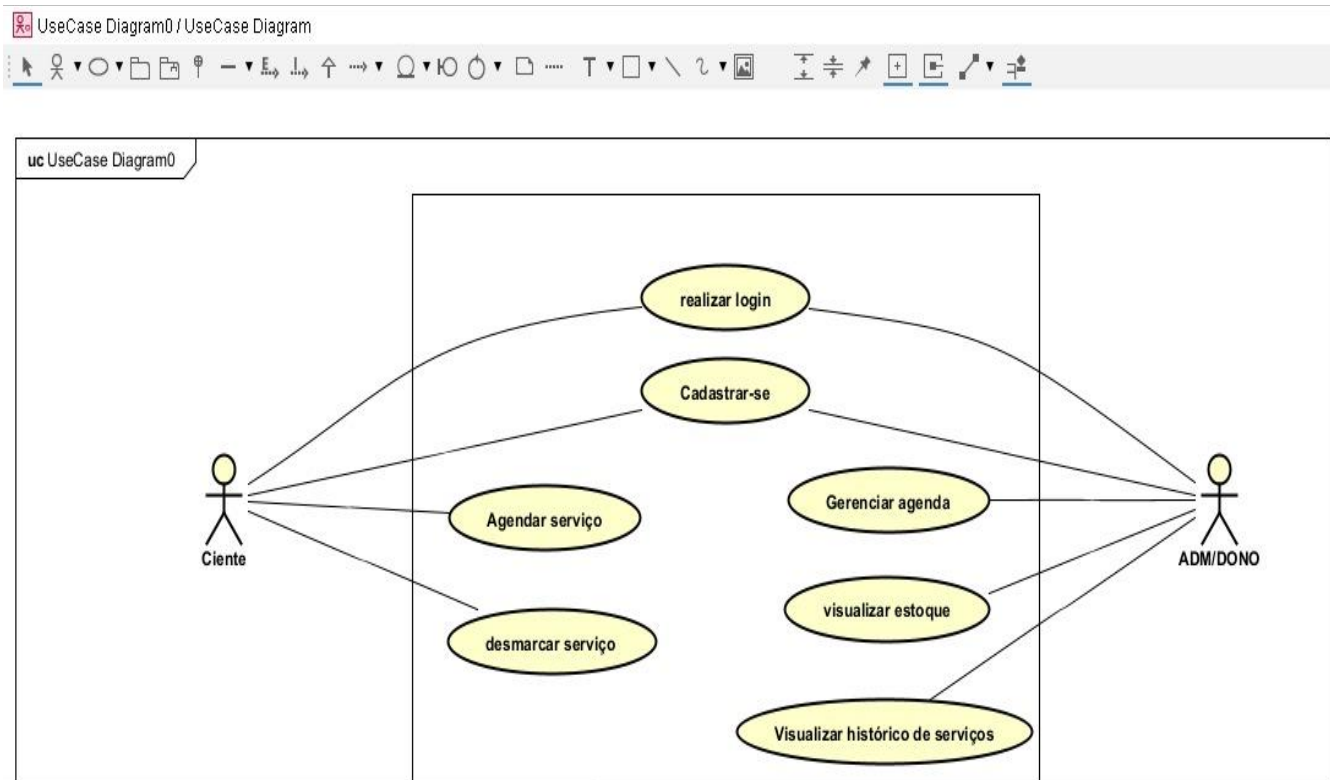
Quadro 1 – Requisitos Funcionais e Não Funcionais (Sprint 1)

Identificador	Tipo	Descrição do Requisito
RF-01	Funcional	O sistema deve permitir o cadastro de novos usuários contendo nome, CPF, e-mail, senha e tipo de perfil.
RF-02	Funcional	O sistema deve autenticar usuários previamente cadastrados solicitando login (CPF/E-mail) e senha.
RF-03	Funcional	O sistema deve gerenciar os níveis de permissão, direcionando o Cliente e o Administrador para menus distintos.
RF-04	Funcional	O sistema deve validar o formato e o preenchimento obrigatório dos campos no momento do cadastro.
RNF-01	Não Funcional	Segurança: As senhas devem possuir um tamanho mínimo de 6 caracteres e não devem ser exibidas em texto puro durante a digitação.
RNF-02	Não Funcional	Desempenho: A verificação de login e a busca de dados na memória do sistema devem ser concluídas em tempo inferior a 1 segundo.
RNF-03	Não Funcional	Usabilidade: A interface no terminal deve apresentar menus numéricos claros, organizados e mensagens amigáveis de retorno.

3.3 Modelagem de Processos com UML

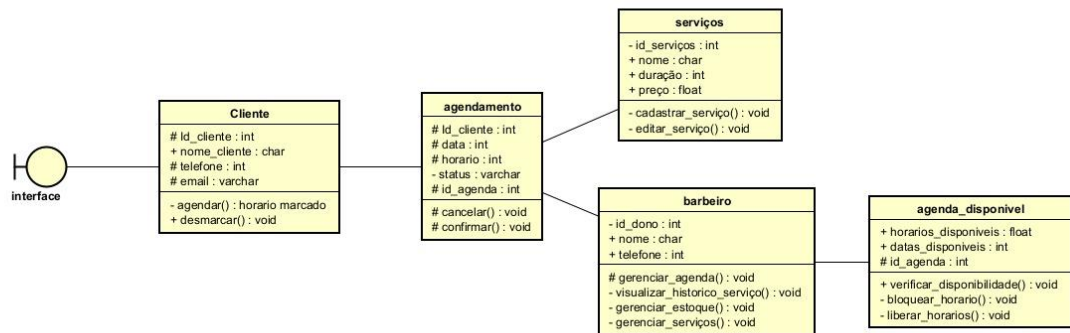
A modelagem orientada a objetos foi estruturada por meio da linguagem UML (Unified Modeling Language) com o auxílio da ferramenta Astah UML. O Diagrama de Casos de Uso da Sprint 1 (Figura 1) mapeia as interações entre os atores do sistema (Cliente e Proprietário/Administrador) e as rotinas de autenticação e gestão de perfil

Figura 1 – Diagrama de Casos de Uso da Sprint 1



Complementarmente, o Diagrama de Classes (Figura 2) define a estrutura lógica dos objetos, contendo a classe Usuario com seus respectivos atributos privativos e métodos públicos de manipulação e validação.

Figura 2 – Diagrama de Classes da Sprint 1

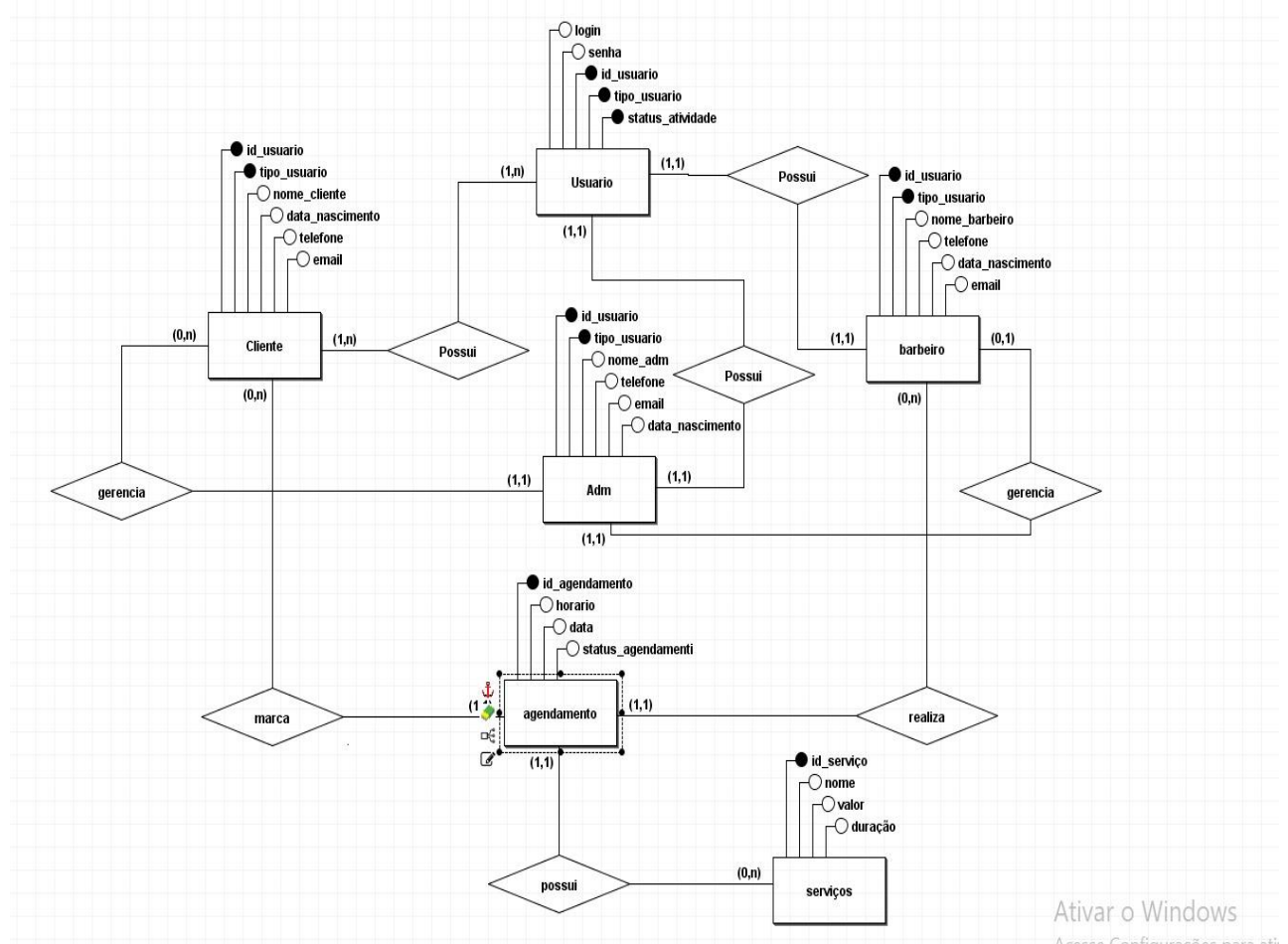


3.4 Modelagem de Banco de Dados (brModelo)

A concepção do repositório de dados para a Barbearia Santos foi realizada no software brModelo. Na Sprint 1, a modelagem conceitual e lógica prioriza a entidade responsável por armazenar as credenciais e perfis dos usuários.

O Modelo Entidade-Relacionamento (MER), apresentado na Figura 3, estabelece a entidade USUARIO, definindo o atributo cpf como chave primária (PK) para garantir a unicidade dos registros, além dos atributos nome, Email, senha e perfil_acesso.

Figura 3 – Modelo Entidade-Relacionamento (MER) para Autenticação



3.5 Implementação do Algoritmo em C (Code::Blocks)

A lógica computacional da Sprint 1 foi desenvolvida no ambiente Code::Blocks utilizando a linguagem C de forma estruturada. Para representar a entidade de usuário em memória, aplicou-se a estrutura de dados struct, combinada com vetores e ponteiros para manipulação dos dados cadastrais.

O trecho de código abaixo ilustra a definição da estrutura Usuario e a função principal de menu interativo para autenticação e cadastro:

Menu Principal da aplicação em C

```
===== BARBEARIA SANTOS =====
1 - Cadastrar
2 - LOGIN
3 - Sair
```

Tela de preenchimento e confirmação do cadastro

```
Digite seu nome: user
Digite seu email: user@unip.br
Digite seu telefone: 1299999999
tamanho correto
Crie uma senha:0000

===== DADOS CADASTRADOS =====
ID: 1
Nome: user
Email: user@unip.br
Telefone: 1299999999
```

Validação de credenciais e autenticação no sistema

```
===== LOGIN =====
Digite seu email:
user@unip.br
Digite sua senha:
0000

Login realizado com sucesso!

===== DADOS DO USUARIO =====
1 - Criar agendamento
2 - Ver agendamento
3 - Excluir agendamento
4 - Sair da conta
_
```

4 CONSIDERAÇÕES FINAIS

O desenvolvimento da primeira etapa do Projeto Integrado Multidisciplinar (PIM II) permitiu a aplicação prática e integrada dos conceitos teóricos abordados ao longo do semestre no curso de Análise e Desenvolvimento de Sistemas. O projeto teve como foco central a resolução dos gargalos operacionais da Barbearia Santos, propondo a transição dos registros manuais e informais para uma solução de software padronizada, segura e funcional.

Nesta Sprint 1, alcançou-se com êxito a estruturação do núcleo de controle do sistema. A etapa compreendeu a especificação detalhada dos Requisitos Funcionais e Não Funcionais, a modelagem de processos através dos diagramas de Casos de Uso e de Classes em UML, a concepção do Modelo Entidade-Relacionamento (MER) no brModelo e a implementação do algoritmo em linguagem C no ambiente Code::Blocks. As rotinas de cadastro, validação de dados, autenticação e separação de permissões entre os perfis de Cliente e Administrador foram testadas e validadas.

Por se tratar de um sistema modular desenvolvido sob a metodologia ágil, os resultados obtidos nesta fase estabelecem a arquitetura de dados e a infraestrutura lógica necessárias para o avanço do projeto. O módulo de autenticação consolidado na Sprint 1 servirá de base direta para a implementação das funcionalidades previstas para as próximas etapas, tais como a gestão de horários e agendamentos em tempo real, o controle automatizado de estoque de produtos e a consulta ao histórico de serviços prestados.

Além dos ganhos técnicos, a condução do trabalho fortaleceu a aplicação de práticas de Engenharia de Software, o versionamento estruturado e o trabalho em equipe. Conclui-se que a Sprint 1 atingiu integralmente seus objetivos propostos, demonstrando o impacto positivo da tecnologia na otimização de rotinas administrativas e reafirmando o compromisso da equipe com o desenvolvimento de uma aplicação robusta e alinhada às necessidades reais do mercado.

5 REFERÊNCIAS

- BEZERRA, Eduardo. Princípios de Análise e Projeto de Sistemas com UML. 3. ed. Rio de Janeiro: Elsevier, 2015.
- DEITEL, Paul; DEITEL, Harvey. C: Como Programar. 6. ed. São Paulo: Pearson Education do Brasil, 2011.
- HEUSER, Carlos Alberto. Projeto de Banco de Dados. 6. ed. Porto Alegre: Bookman, 2009.
- PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 8. ed. Porto Alegre: AMGH, 2016.
- SANTOS, Rafael. brModelo: Ferramenta Acadêmica para Modelagem de Banco de Dados Relacional. Disponível em: <<http://www.sis4.com/brmodelo/>>. Acesso em: 04 out. 2026.

6 APÊNDICE – Código-Fonte Completo da Aplicação em C

Início do programa, declaração de variáveis e menu principal (linhas 1 a 30)

```
1 // cadastro teste
2 #include <stdio.h>
3 #include <string.h>
4
5 int main() {
6     //char
7     char nome[10][50];
8     char email[10][100];
9     char telefone[10][20];
10    char senha[10][50];
11    char data[10][11];
12    char horario[10][6];
13
14    //int
15    int ids[10];
16    int quantidade = 0;
17    int opcao = 0;
18    int agendamento[10] = {0};
19
20
21    char emailLogin[100];
22    char senhaLogin[50];
23
24    while (opcao != 3) { // tela de inicio
25        printf("\n===== BARBEARIA SANTOS =====\n");
26        printf("1 - Cadastrar\n");
27        printf("2 - LOGIN\n");
28        printf("3 - Sair\n");
29
30        if (scanf("%d", &opcao) != 1) { // verifica a opcao 1, 2 ou 3
```

Lógica de cadastro do usuário e validação de e-mail (linhas 36 a 70)

```
36    if (quantidade >= 10) { // verifica o limite de cadastros maximo 10
37        printf("Limite de cadastros atingido!\n");
38        continue;
39    }
40
41    printf("\nDigite seu nome: ");
42    scanf("%49[^\n]", nome[quantidade]);
43
44    ids[quantidade] = quantidade + 1; // id de identificação do numero do usuario
45
46    do{
47        printf("\nDigite seu email: "); // o email so e aceito se tiver arroba como certificacao de email
48        scanf("%99s", email[quantidade]);
49        int posicaoarroba = -1;
50        for(int i; i < strlen(email[quantidade]); i++){
51            if (email[quantidade][i] == '@') {
52                posicaoarroba = i;
53            }
54        }
55        if (strchr(email[quantidade], '@') == NULL) { // verifica se tem o arroba
56            printf("Faltou o @ no seu email.\n");
57        }
58    }while (strchr(email[quantidade], '@') == NULL);
59
60    int telefonevalido = 1;
61
62    do {
63        telefonevalido = 1; // o telefone sera valido apenas se tiver entre 10 a 11 numeros
64        printf("\nDigite seu telefone: ");
65        scanf("%19s", telefone[quantidade]);
66        int tamanhotelefone = strlen(telefone[quantidade]);
67        if (tamanhotelefone == 10 || tamanhotelefone == 11){
68            printf("tamanho correto\n");
69        } else {
70            telefonevalido = 0;
```

Validação de telefone, criação de senha e tela de login (linhas 71 a 105)

```

71         printf("O telefone deve ter 10 ou 11 digitos.\n");
72     }
73
74
75     for (int i = 0; i < tamanhotelefone; i++){
76         if (telefone[quantidade][i] < '0' ||
77             telefone[quantidade][i] > '9') {
78             telefonevalido = 0;
79             printf("Caractere invalido!\n");
80         }
81     }
82 } while (telefonevalido == 0);
83
84
85     printf("\nCrie uma senha:"); // a senha podera ser criada com um limite de 49 caracteres
86     scanf("%49s", senha[quantidade]);
87
88     quantidade++;
89
90     printf("\n==== DADOS CADASTRADOS =====\n"); // tela dos dados do usuario
91     printf("ID: %d\n", ids[quantidade - 1]);
92     printf("Nome: %s\n", nome[quantidade - 1]);
93     printf("Email: %s\n", email[quantidade - 1]);
94     printf("Telefone: %s\n", telefone[quantidade - 1]);
95
96 } else if (opcao == 2) {
97     printf("\n==== LOGIN =====\n"); // tela de login
98
99     printf("Digite seu email:\n");
100    scanf("%99s", emailLogin);
101
102    printf("Digite sua senha:\n");
103    scanf("%49s", senhaLogin);
104
105    int loginValido = 0;

```

Autenticação de credenciais e menu de agendamentos (linhas 105 a 139)

```

105    int loginValido = 0;
106    int usuariologado = -1;
107
108    for (int i = 0; i < quantidade; i++) {
109        if (strcmp(emailLogin, email[i]) == 0 &&
110            strcmp(senhaLogin, senha[i]) == 0) {
111
112            loginValido = 1;
113            usuariologado = i;
114
115            printf("\nLogin realizado com sucesso!\n"); // verificacao realizada com sucesso
116            int opcaoUsuario = 0;
117
118            while (opcaoUsuario != 4) {
119                printf("\n==== DADOS DO USUARIO =====\n"); //tela para criacao, verificacao ou exclusao de agendamentos
120                printf("1 - Criar agendamento\n");
121                printf("2 - Ver agendamento\n");
122                printf("3 - Excluir agendamento\n");
123                printf("4 - Sair da conta\n");
124
125                if (scanf("%d", &opcaoUsuario) != 1) {
126                    printf("Entrada invalida!\n");
127                    return 1;
128                }
129
130                if (opcaoUsuario == 1) { // criacao de agendamento data, hora e confirmacao
131                    if (agendamento[usuariologado] == 0) {
132                        printf("Digite a data (DD/MM/AAAA):\n");
133                        scanf("%10s", data[usuariologado]);
134                        printf("Digite um horario\n");
135                        scanf("%5s", horario[usuariologado]);
136                        agendamento[usuariologado] = 1;
137                        printf("Agendamento criado!\n");
138                    } else {
139                        printf("Voce ja possui um agendamento!\n");

```

Visualização, exclusão de agendamento e encerramento do sistema (linhas 138 a 172)

```

138         } else {
139             printf("Voce ja possui um agendamento!\n");
140         }
141     } else if (opcaoUsuario == 2) { // ver horario e data do agendamento do usuario
142         if (agendamento[usuariologado] == 1) {
143             printf("seu agendamento e: %s\n", data[usuariologado], horario[usuariologado]);
144         } else {
145             printf("voce nao possui um agendamento\n");
146         }
147     } else if (opcaoUsuario == 3) { // exclusao do agendamento do usuario
148         if (agendamento[usuariologado] == 1) {
149             agendamento[usuariologado] = 0;
150             printf("agendamento excluido com sucesso\n");
151         }
152     }
153 }
154
155     break; // encerra a busca
156 }
157
158
159     if (loginValido == 0) {
160         printf("\nEmail ou senha incorretos!\n");
161     }
162
163 } else if (opcao == 3) {
164     printf("\nSaindo do sistema...\n");
165 } else {
166     printf("\nOpcao invalida!\n");
167 }
168
169
170 return 0;
171 }
172

```